Verilog Assignments

Ajaya Kumar Patel
Roll No: CrS2503

1 Assignment 1: Full Adder Design

1.1 Problem Statement

Write the Verilog code for a Full Adder that takes three 1-bit inputs `a`, `b`, and `carryin`, and gives 1-bit outputs `sum` and `carryout`. Write a testbench that applies all possible input combinations.

1.2 Design Code

```
module full_adder (
    input a,
    input b,
    input carryin,
    output sum,
    output carryout
);

assign {carryout, sum} =
       {1'b0, a} + {1'b0, b} + carryin;

endmodule
```

1.3 Testbench Code

```
module tb_full_adder;

reg a, b, carryin;
wire sum, carryout;

reg [1:0] expected;
integer i;

full_adder uut (
    .a(a),
    .b(b),
    .carryin(carryin),
    .sum(sum),
    .carryout(carryout)
);

initial begin
    $display("Full Adder Verification");
```

```
    $display("-------------------------------");
    $display("a b cin | sum cout | Status");
    $display("-------------------------------");

    for (i = 0; i < 8; i = i + 1) begin

        {a, b, carryin} = i;
        #10;

        expected = {1'b0, a} + {1'b0, b} + carryin;

        if ({carryout, sum} === expected)
            $display("%b %b  %b  |  %b   %b   | PASS",
                     a, b, carryin, sum, carryout);
        else
            $display("%b %b  %b  |  %b   %b   | FAIL",
                     a, b, carryin, sum, carryout);

    end

    $finish;
end

endmodule
```

1.4 Simulation Output

a	b	carryin	sum	carryout	Status
0	0	0	0	0	PASS
0	0	1	1	0	PASS
0	1	0	1	0	PASS
0	1	1	0	1	PASS
1	0	0	1	0	PASS
1	0	1	0	1	PASS
1	1	0	0	1	PASS
1	1	1	1	1	PASS

2 Assignment 2: Multiplexer Design

2.1 Problem Statement

Write the Verilog code for a Multiplexer that takes in two 1-bit data inputs (`datain_0`, `datain_1`) and one select line (`sel`), producing a 1-bit output `mux_out`. Write a testbench to test all possible combinations.

2.2 Design Code

```
`timescale 1ns/1ps

module multiplexer (
    input datain_0,
    input datain_1,
    input sel,
    output reg mux_out
);

always @(*) begin
    case (sel)
        1'b0: mux_out = datain_0;
        1'b1: mux_out = datain_1;
    endcase
end

endmodule
```

2.3 Testbench Code

```
`timescale 1ns/1ps

module tb_multiplexer;

reg datain_0, datain_1, sel;
wire mux_out;

reg expected;
integer i;

multiplexer uut (
    .datain_0(datain_0),
    .datain_1(datain_1),
    .sel(sel),
    .mux_out(mux_out)
);

initial begin
    $display("Multiplexer Verification");
    $display("--------------------------------");
    $display("D0 D1 Sel | Output | Status");
    $display("--------------------------------");

    for (i = 0; i < 8; i = i + 1) begin

```

```
        {datain_0, datain_1, sel} = i;
        #10;

        expected = sel ? datain_1 : datain_0;

        if (mux_out === expected)
            $display("%b  %b   %b  |   %b    | PASS",
                     datain_0, datain_1, sel, mux_out);
        else
            $display("%b  %b   %b  |   %b    | FAIL",
                     datain_0, datain_1, sel, mux_out);

    end

    $finish;
end

endmodule
```

2.4 Simulation Output

datain_0	datain_1	sel	mux_out	Status
0	0	0	0	PASS
0	0	1	0	PASS
0	1	0	0	PASS
0	1	1	1	PASS
1	0	0	1	PASS
1	0	1	0	PASS
1	1	0	1	PASS
1	1	1	1	PASS

3 Assignment 3: Decoder Design Using Gate Primitives

3.1 Problem Statement

Design a 2-to-4 decoder using Verilog gate primitives and verify it with a comprehensive testbench.

3.2 Design Code

```
`timescale 1ns/1ps

module decoder_2to4 (
    input a,
    input b,
    output y0,
    output y1,
    output y2,
    output y3
);

wire a_bar, b_bar;
wire n0, n1, n2, n3;

not (a_bar, a);
not (b_bar, b);

nand (n0, a_bar, b_bar);
nand (n1, a_bar, b);
nand (n2, a, b_bar);
nand (n3, a, b);

not (y0, n0);
not (y1, n1);
not (y2, n2);
not (y3, n3);

endmodule
```

3.3 Testbench Code

```
`timescale 1ns/1ps

module tb_decoder_2to4;

reg a, b;
wire y0, y1, y2, y3;

reg [3:0] expected;
integer i;

decoder_2to4 uut (
    .a(a),
    .b(b),
    .y0(y0),
    .y1(y1),
```

```
    .y2(y2),
    .y3(y3)
);

initial begin
    $display("2-to-4 Decoder Verification");
    $display("--------------------------------");
    $display("a b | y0 y1 y2 y3 | Status");
    $display("--------------------------------");

    for (i = 0; i < 4; i = i + 1) begin

        {a, b} = i;
        #10;

        case ({a, b})
            2'b00: expected = 4'b1000;
            2'b01: expected = 4'b0100;
            2'b10: expected = 4'b0010;
            2'b11: expected = 4'b0001;
        endcase

        if ({y0, y1, y2, y3} === expected)
            $display("%b %b | %b  %b  %b  %b | PASS",
                     a, b, y0, y1, y2, y3);
        else
            $display("%b %b | %b  %b  %b  %b | FAIL",
                     a, b, y0, y1, y2, y3);

    end

    $finish;
end

endmodule
```

3.4 Simulation Output

a	b	y0	y1	y2	y3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

4 Assignment 4: Decoder Design Using Assign Statements

4.1 Problem Statement

Design a 2-to-4 decoder using continuous `assign` statements and verify it with a complete testbench.

4.2 Design Code

```
`timescale 1ns/1ps

module decoder_2to4_assign (
    input a,
    input b,
    output y0,
    output y1,
    output y2,
    output y3
);

assign y0 = ~(a | b);
assign y1 = ~(a | ~b);
assign y2 = ~(~a | b);
assign y3 = a & b;

endmodule
```

4.3 Testbench Code

```
`timescale 1ns/1ps

module tb_decoder_2to4_assign;

reg a, b;
wire y0, y1, y2, y3;

reg [3:0] expected;
integer i;

decoder_2to4_assign uut (
    .a(a),
    .b(b),
    .y0(y0),
    .y1(y1),
    .y2(y2),
    .y3(y3)
);

initial begin
    $display("Decoder Using Assign Verification");
    $display("---------------------------------");
    $display("a b | y0 y1 y2 y3 | Status");
    $display("---------------------------------");

    for (i = 0; i < 4; i = i + 1) begin
```

```
27
28          {a, b} = i;
29          #10;
30
31          case ({a, b})
32              2'b00: expected = 4'b1000;
33              2'b01: expected = 4'b0100;
34              2'b10: expected = 4'b0010;
35              2'b11: expected = 4'b0001;
36          endcase
37
38          if ({y0, y1, y2, y3} === expected)
39              $display("%b %b | %b  %b  %b  %b | PASS",
40                       a, b, y0, y1, y2, y3);
41          else
42              $display("%b %b | %b  %b  %b  %b | FAIL",
43                       a, b, y0, y1, y2, y3);
44
45      end
46
47      $finish;
48  end
49
50  endmodule
```

4.4 Simulation Output

a	b	y0	y1	y2	y3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

5 Assignment 5: Complex Multiplexer Design

5.1 Problem Statement

Write the Verilog code for a Multiplexer that takes a 4-bit data input and a 2-bit select input, producing a 1-bit output using array indexing concepts.

5.2 Design Code

```
`timescale 1ns/1ps

module complex_multiplexer (
    input [3:0] data,
    input [1:0] sel,
    output mux_out
);

assign mux_out = data[sel];

endmodule
```

5.3 Testbench Code

```
`timescale 1ns/1ps

module tb_complex_multiplexer;

reg [3:0] data;
reg [1:0] sel;

wire mux_out;
reg expected;

integer i, j;

complex_multiplexer uut (
    .data(data),
    .sel(sel),
    .mux_out(mux_out)
);

initial begin
    $display("4-to-1 Multiplexer Verification");
    $display("-----------------------------------");
    $display("Data  Sel | Output | Status");
    $display("-----------------------------------");

    for (i = 0; i < 16; i = i + 1) begin

        for (j = 0; j < 4; j = j + 1) begin

            data = i;
            sel = j;
            #10;

```

```
            expected = data[sel];

            if (mux_out === expected)
                $display("%b  %b |   %b    | PASS",
                         data, sel, mux_out);
            else
                $display("%b  %b |   %b    | FAIL",
                         data, sel, mux_out);

        end
    end

    $finish;
end

endmodule
```

5.4 Simulation Output

The testbench checks all 16 possible data combinations and all 4 select-line combinations, resulting in 64 test cases.

Data	Sel	mux_out	Status
0000	00	0	PASS
0001	00	1	PASS
0010	01	1	PASS
0100	10	1	PASS
1000	11	1	PASS
1111	00	1	PASS
1111	01	1	PASS
1111	10	1	PASS
1111	11	1	PASS

6 Assignment 6: Gray to Binary Converter

6.1 Problem Statement

Design a combinational circuit named `GraytoBin` for converting a 4-bit Gray code to its equivalent binary number using exclusive-OR operations.

Let b_0, b_1, b_2, b_3 be the binary bits (LSB to MSB) and g_0, g_1, g_2, g_3 be the Gray code bits.

6.2 Design Code

```
`timescale 1ns/1ps

module GraytoBin (
    input [3:0] gray,
    output [3:0] bin
);

assign bin[3] = gray[3];
assign bin[2] = bin[3] ^ gray[2];
assign bin[1] = bin[2] ^ gray[1];
assign bin[0] = bin[1] ^ gray[0];

endmodule
```

6.3 Testbench Code

```
`timescale 1ns/1ps

module tb_GraytoBin;

reg [3:0] gray;
wire [3:0] bin;

reg [3:0] expected;
integer i;

GraytoBin uut (
    .gray(gray),
    .bin(bin)
);

initial begin
    $display("Gray to Binary Verification");
    $display("-------------------------------");
    $display("Gray | Binary | Status");
    $display("-------------------------------");

    for (i = 0; i < 16; i = i + 1) begin

        gray = i;
        #10;

        expected[3] = gray[3];
        expected[2] = expected[3] ^ gray[2];
        expected[1] = expected[2] ^ gray[1];
```

```
        expected[0] = expected[1] ^ gray[0];

        if (bin === expected)
            $display("%b  |  %b   | PASS",
                     gray, bin);
        else
            $display("%b  |  %b   | FAIL",
                     gray, bin);

    end

    $finish;
end

endmodule
```

6.4 Simulation Output

Gray Code	Binary Output	Status
0000	0000	PASS
0001	0001	PASS
0010	0011	PASS
0011	0010	PASS
0100	0111	PASS
0101	0110	PASS
0110	0100	PASS
0111	0101	PASS
1000	1111	PASS
1001	1110	PASS
1010	1100	PASS
1011	1101	PASS
1100	1000	PASS
1101	1001	PASS
1110	1011	PASS
1111	1010	PASS

7 Assignment 7: Latch and Flip-flop Design

7.1 Problem Statement

Design various D flip-flops (rising/falling edge, simple, synchronous reset, asynchronous reset active high/low) and a transparent D latch in Verilog, accompanied by a comprehensive testbench.

7.2 Design Code

```
`timescale 1ns/1ps

// Rising-edge triggered D flip-flop
module dff_rise_simple (
    input d,
    input clk,
    output reg q
);

always @(posedge clk)
    q <= d;

endmodule


// Falling-edge triggered D flip-flop
module dff_fall_simple (
    input d,
    input clk,
    output reg q
);

always @(negedge clk)
    q <= d;

endmodule


// Rising-edge D flip-flop with synchronous reset
module dff_rise_sync_rst (
    input d,
    input clk,
    input reset,
    output reg q
);

always @(posedge clk) begin
    if (reset)
        q <= 0;
    else
        q <= d;
end

endmodule


// Rising-edge D flip-flop with asynchronous active-high reset
```

```
module dff_async_rst_high (
    input d,
    input clk,
    input reset,
    output reg q
);

always @(posedge clk or posedge reset) begin
    if (reset)
        q <= 0;
    else
        q <= d;
end

endmodule


// Rising-edge D flip-flop with asynchronous active-low reset
module dff_async_rst_low (
    input d,
    input clk,
    input reset_n,
    output reg q
);

always @(posedge clk or negedge reset_n) begin
    if (!reset_n)
        q <= 0;
    else
        q <= d;
end

endmodule


// Level-sensitive transparent D latch
module d_latch (
    input d,
    input en,
    output reg q
);

always @(*) begin
    if (en)
        q = d;
end

endmodule
```

7.3 Testbench Code

```
`timescale 1ns/1ps

module tb_flipflops;

reg d, clk, reset, reset_n, en;

```

```
wire q_rise;
wire q_fall;
wire q_sync;
wire q_async_high;
wire q_async_low;
wire q_latch;

// Rising-edge flip-flop
dff_rise_simple u1 (
    .d(d), .clk(clk), .q(q_rise)
);

// Falling-edge flip-flop
dff_fall_simple u2 (
    .d(d), .clk(clk), .q(q_fall)
);

// Synchronous reset flip-flop
dff_rise_sync_rst u3 (
    .d(d), .clk(clk), .reset(reset), .q(q_sync)
);

// Asynchronous active-high reset
dff_async_rst_high u4 (
    .d(d), .clk(clk), .reset(reset), .q(q_async_high)
);

// Asynchronous active-low reset
dff_async_rst_low u5 (
    .d(d), .clk(clk), .reset_n(reset_n),
    .q(q_async_low)
);

// Transparent D latch
d_latch u6 (
    .d(d), .en(en), .q(q_latch)
);

// Clock generation
initial clk = 0;
always #5 clk = ~clk;

// Input stimulus and monitoring
initial begin

    d = 0;
    reset = 0;
    reset_n = 1;
    en = 0;

    $display("Flip-flop and Latch Simulation");

    $monitor(
        "Time=%0t | CLK=%b D=%b Reset=%b Reset_n=%b EN=%b | Rise=%b Fall=%b Sync=%b Async_H=%b Async_L=%b Latch=%b",
        $time, clk, d, reset, reset_n, en,
        q_rise, q_fall, q_sync, q_async_high,
        q_async_low, q_latch
```

```
    );

    #2 d = 1;
    #10 d = 0; en = 1;
    #5 d = 1;
    #3 en = 0;
    #2 reset = 1;
    #10 reset = 0;
    #5 reset_n = 0;
    #10 reset_n = 1;
    #20 $finish;

end

endmodule
```

7.4 Expected Behaviour

Circuit	Trigger / Condition	Operation
Rising-edge DFF	Positive clock edge	Stores input D
Falling-edge DFF	Negative clock edge	Stores input D
Synchronous reset DFF	Positive clock edge	Resets when reset = 1
Async active-high DFF	Reset rising edge	Resets immediately
Async active-low DFF	Reset falling edge	Resets immediately
D latch	Enable = 1	Output follows input D

References

1. Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd Edition, Prentice Hall, 2003.
2. M. Morris Mano and Michael D. Ciletti, *Digital Design*, 5th Edition, Pearson, 2013.
3. OpenAI, *ChatGPT: Assistance with Verilog code development and documentation*. Available at: https://chatgpt.com/